# COMP9334: Capacity Planning of Computer Systems and Networks

Optimisation (3):
Network flow

# The story so far

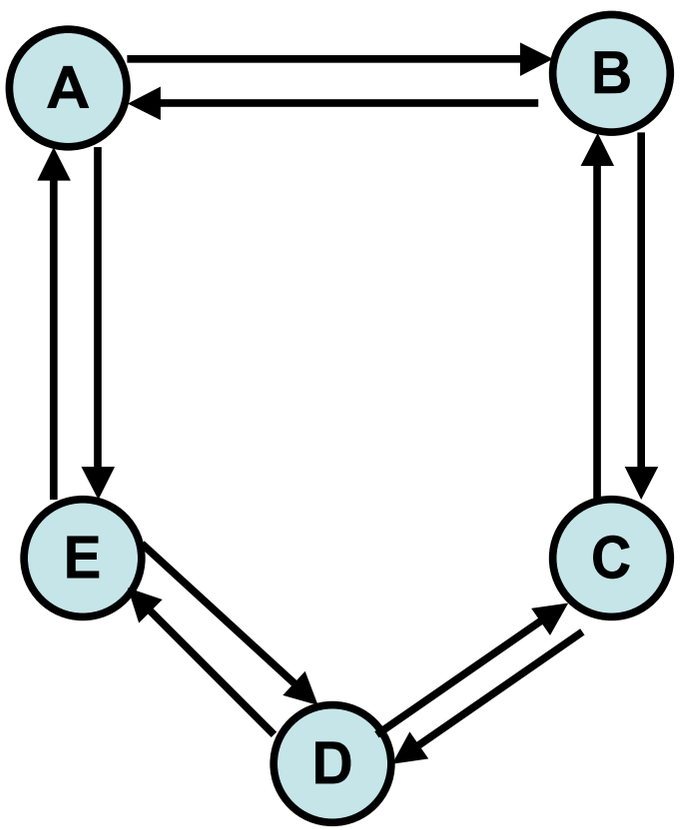
- Linear programming (LP)
  - Real values for decision variables, linear in objective function, linear in constraints
  - Large LP problems can be solved routinely
- Integer programming (IP)
  - Some decision variables can only take integer values
  - Some decision variables can only take binary values, e.g. for making yes-or-no decisions
  - IP problems can be solved using branch and bound in principle
  - Computation complexity is generally exponential except for unimodular problems
  - Use linear integer programming whenever possible

# Week 9B

- Applications of integer programming for network flow problems
  - Traffic Engineering
  - Dimensioning problem
  - Topology design

# Traffic Engineering Example (1)

An ISP owns the following network which connects
5 cities A, B, C, D and E.
Capacity of each link is 1000 Mbps

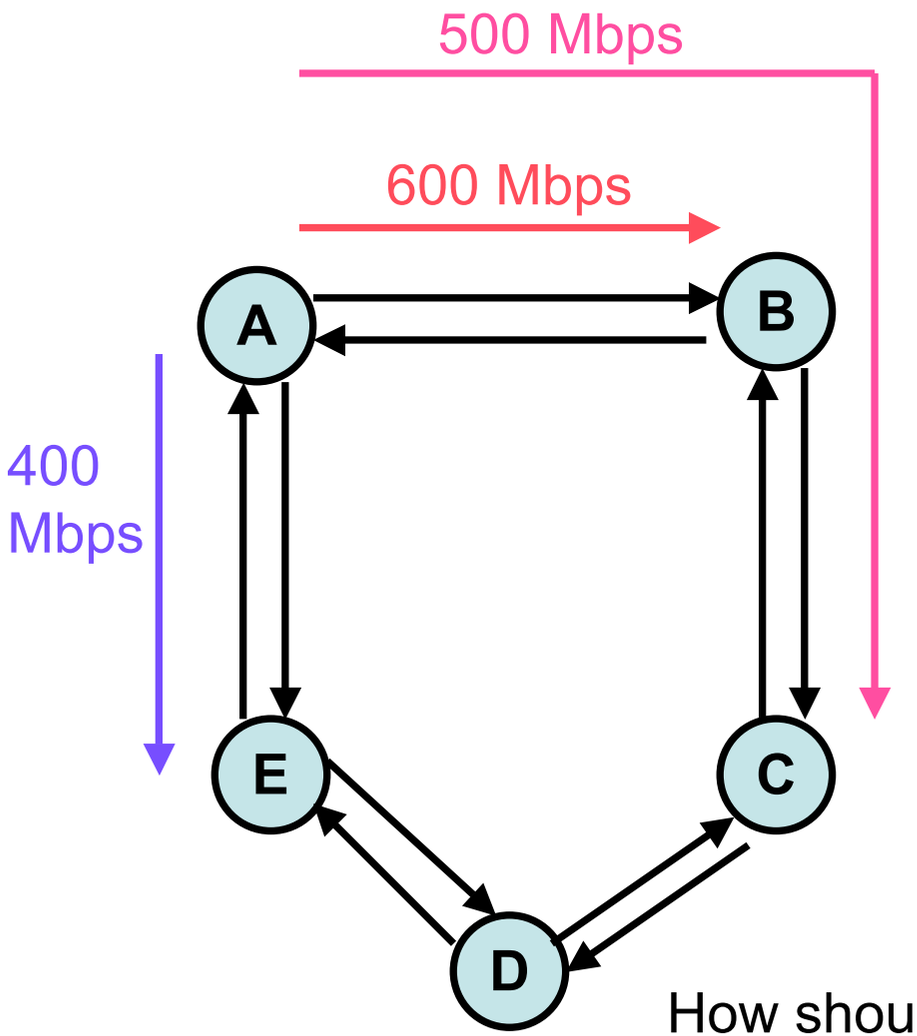



The traffic demands between cities are:
A to B: 600 Mbps
A to E: 400 Mbps
A to C: 500 Mbps

Question: How should we route the traffic so that the links are at most 80% utilised and we use the minimum amount of resources?

# Traffic Engineering Example (2)

Capacity of each link is 1000 Mbps




The traffic demands between cities are:
A to B: 600 Mbps
A to E: 400 Mbps
A to C: 500 Mbps

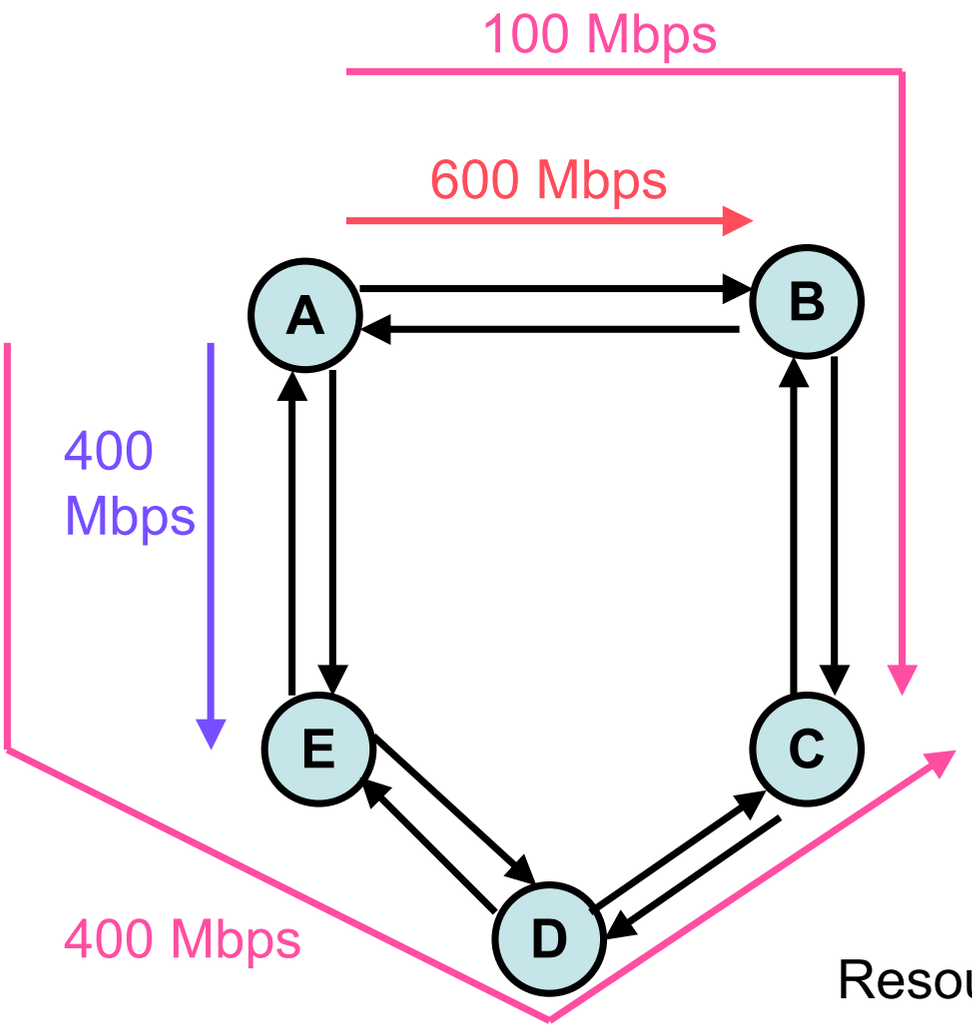
Let us assume that the traffic demand will take the shortest path between its end points

But 1100 Mbps in link A-B! The link is over-utilised!

How should you route the traffic?

# Traffic Engineering Example (3)

Capacity of each link is 1000 Mbps




- The traffic demands between cities are:
  - A to B: 600 Mbps
  - A to E: 400 Mbps
  - A to C: 500 Mbps

- Traffic in links
  - A-B = 700 Mbps
  - B-C = 100 Mbps
  - A-E = 800 Mbps
  - E-D = 400 Mbps
  - D-C = 400 Mbps

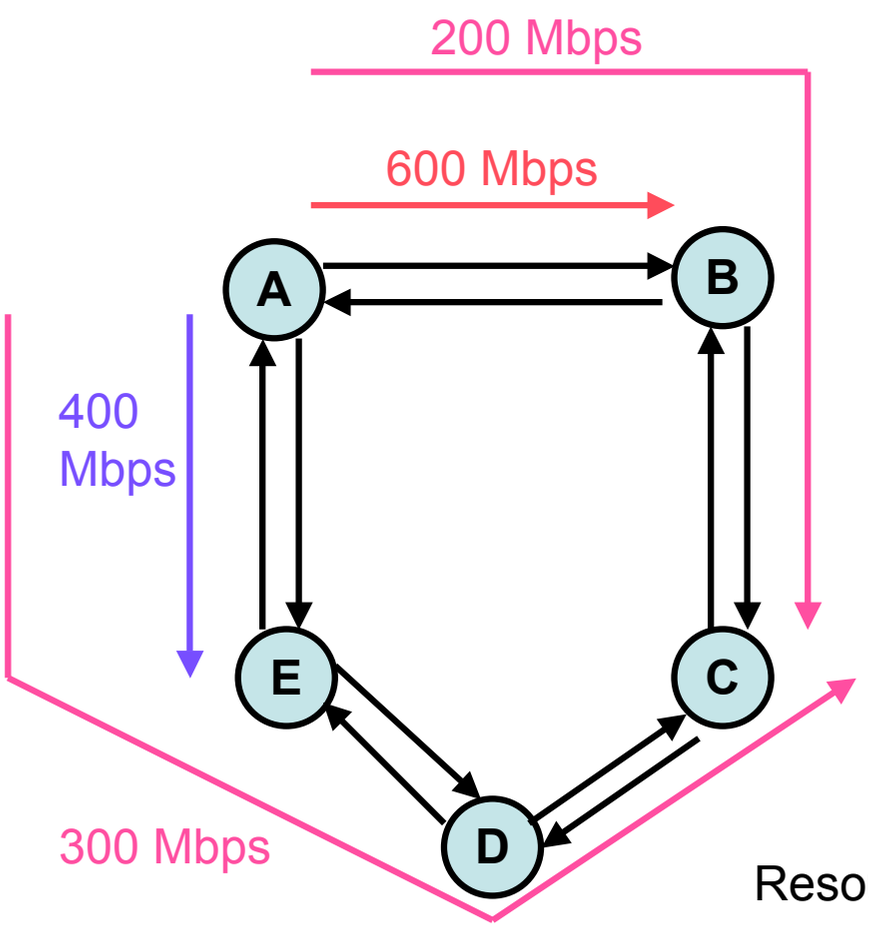
} +

- Link A-E is 80% utilised. Others are less utilised.

Resources used = 2400Mbps

# Traffic Engineering Example (4)

Capacity of each link is 1000 Mbps




- The traffic demands between cities are:
  - A to B: 600 Mbps
  - A to E: 400 Mbps
  - A to C: 500 Mbps

- Traffic in links
  - A-B = 800 Mbps
  - B-C = 200 Mbps
  - A-E = 700 Mbps
  - E-D = 300 Mbps
  - D-C = 300 Mbps

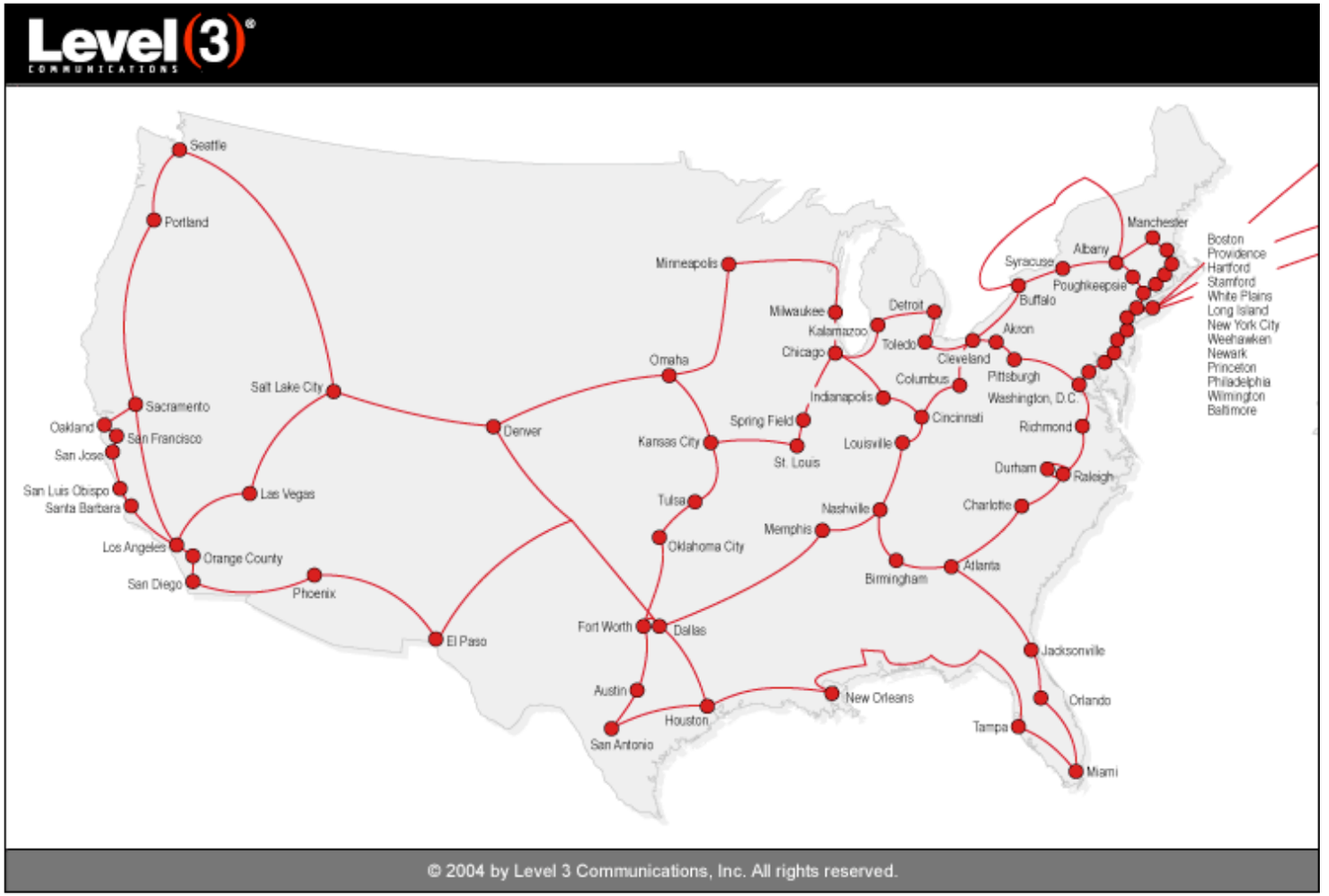
} +

- Link A-B is 80% utilised. Others are less utilised.

Resources used = 2300Mbps

# Traffic Engineering

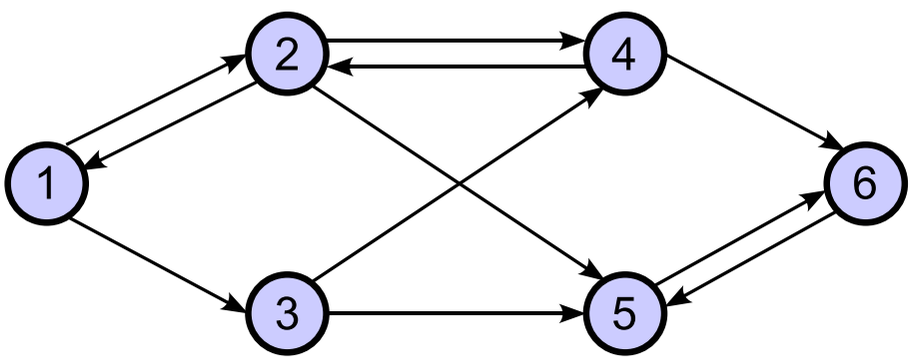
- General traffic engineering problem:
  - Given:
    - A network (i.e. nodes, links and their capacites)
    - The traffic demand between each pair of nodes.
  - Find: how to route the traffic to best utilise the resource
- The traffic engineering example earlier was simple, but for a commercial carrier (Next slide shows the network map of a commercial carrier.), it's no longer so.
  - Traffic engineering problems can be solved systematically using integer programming
  - These problems are generally known as network flow problems. *Note: flow is synonymous with traffic demand between a pair of nodes.*
  - We will start with the simplest network flow problem, finding the shortest path for one flow.


Level(3)
COMMUNICATIONS
Seattle
Portland
Sacramento
Oakland
San Francisco
San Jose
San Luis Obispo
Santa Barbara
Los Angeles
Orange County
San Diego
Phoenix
Las Vegas
Salt Lake City
Denver
El Paso
Minneapolis
Omaha
Kansas City
Tulsa
Oklahoma City
Fort Worth
Dallas
Austin
Houston
San Antonio
Milwaukee
Chicago
Kalamazoo
Detroit
Toledo
Cleveland
Akron
Buffalo
Syracuse
Albany
Manchester
Poughkeepsie
Boston
Providence
Hartford
Stamford
White Plains
Long Island
New York City
Weehawken
Newark
Princeton
Philadelphia
Wilmington
Baltimore
Columbus
Pittsburgh
Washington, D.C.
Indianapolis
Spring Field
St. Louis
Cincinnati
Louisville
Richmond
Durham
Raleigh
Charlotte
Nashville
Memphis
Birmingham
Atlanta
Jacksonville
Orlando
Tampa
Miami
New Orleans
© 2004 by Level 3 Communications, Inc. All rights reserved.

# Network flow problems

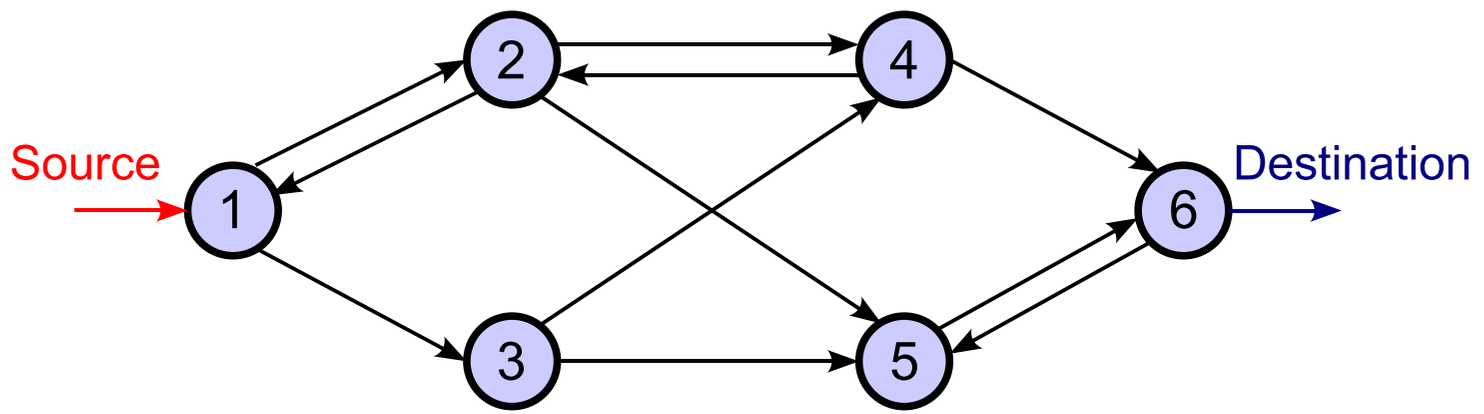
- Network flow problems are important applications of integer programming
  - Move some entity from one point to another in the network
  - Given alternative ways, find the most efficient one, e.g. minimum cost, maximum profit, etc.

- Network is represented as a directed graph $G = (N, E)$
  - $N$ = the set of nodes, e.g. $N = \{1, 2, 3, 4, 5, 6\}$
  - $E$ = the set of directed edges, e.g. $E = \{\langle 1, 2\rangle, \langle 1, 3\rangle, \langle 2, 1\rangle, \ldots\}$



# Finding the shortest path

- Aim: Find the shortest path from the source node to the destination node of a flow




- Cost is generally assumed to be additive, i.e. cost of a path = sum of the cost of using each edge in the path
  - E.g. cost of using edges $\langle 1, 2 \rangle$, $\langle 2, 4 \rangle$ and $\langle 4, 6 \rangle$
    = cost of edge $\langle 1, 2 \rangle$ + cost of edge $\langle 2, 4 \rangle$ + cost of edge $\langle 4, 6 \rangle$

# Shortest path problem (SPP)

- Given
  - A directed graph $G = (N, E)$
  - A flow of size $1$ enters at node $s$ (source) and leaves at node $d$ (destination)
  - It costs $c_{i,j}$ for using directed edge $\langle i, j \rangle$
- Find which directed edges the flow should use in order that
  - The total cost is minimized
  - The entire flow must use only one path
- Logical decision: Should I use a directed edge or not?

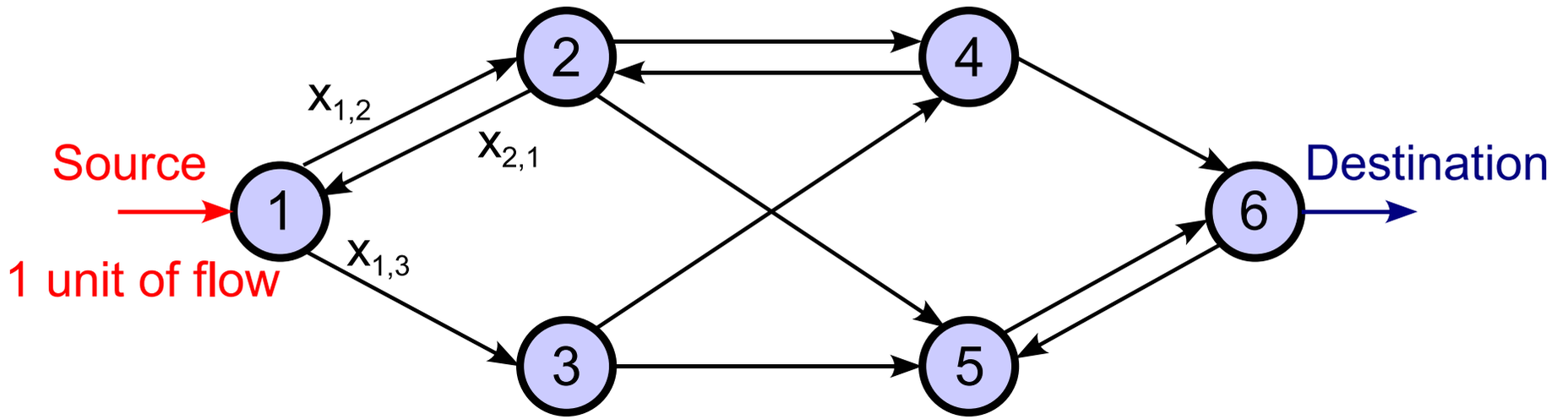
# Formulating SPP

- Decision variables

$$x_{i,j} = \begin{cases} 1 & \text{if directed edge } \langle i, j \rangle \text{ in } E \text{ is used} \\ 0 & \text{otherwise} \end{cases}$$

- We assume 1 unit of flow from the source to the destination
- An important part of the formulation is to make sure the directed edges selected actually form a connected path from the source to the destination
  - This is by adding the conservation of flow constraints

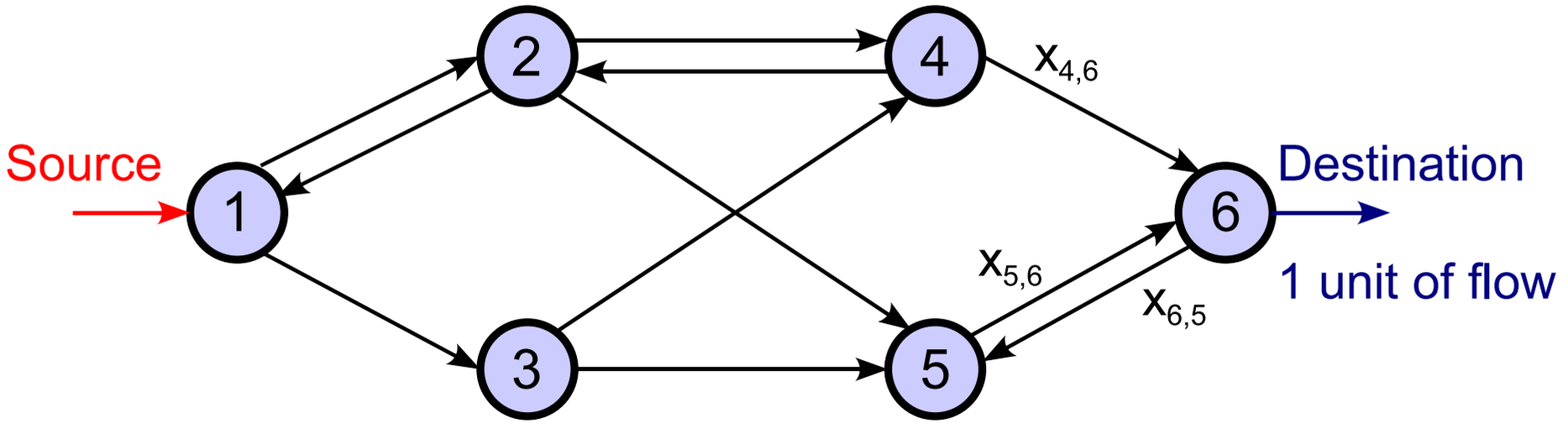
# Conservation of flow: Source node




- What goes in = What goes out
  - Flow going into node 1 from external = 1
  - Flow going into node 1 from neighboring nodes = $x_{2,1}$
    - Second index is “1”
  - Flow going from node 1 to neighboring nodes = $x_{1,2} + x_{1,3}$
    - First index is “1”
  - Therefore, we have

$$1 + x_{2,1} = x_{1,2} + x_{1,3}$$

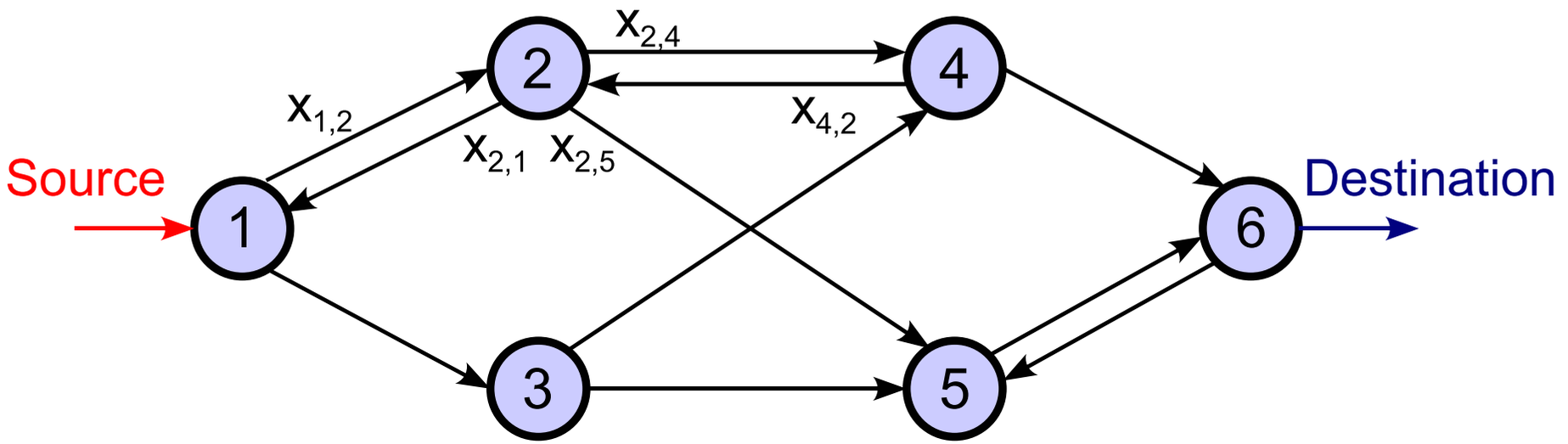
# Conservation of flow: Destination node




- What goes in = What goes out
  - Flow going into node 6 from neighboring nodes = $x_{4,6} + x_{5,6}$
    - Second index is “6”
  - Flow going from node 6 to neighboring nodes = $x_{6,5}$
    - First index is “6”
  - Flow going from node 6 to external = 1
  - Therefore, we have

$$x_{4,6} + x_{5,6} = x_{6,5} + 1$$

# Conservation of flow: Other nodes




- E.g. flow conservation at node 2: What goes in = What goes out
  - Flow going into node 2 from neighboring nodes = $x_{1,2} + x_{4,2}$
    - Second index is “2”
  - Flow going from node 2 to neighboring nodes = $x_{2,1} + x_{2,4} + x_{2,5}$
    - First index is “2”
  - Therefore, we have

$$x_{1,2} + x_{4,2} = x_{2,1} + x_{2,4} + x_{2,5}$$

# Conservation of flow constraints

- In our example network, the source is node 1, so the constraint is

$$1 + x_{2,1} = x_{1,2} + x_{1,3}$$

  - This can be rewritten as

$$\sum_{j:\langle 1,j\rangle \in E} x_{1,j} - \sum_{j:\langle j,1\rangle \in E} x_{j,1} = 1$$

- The destination is node 6, so the constraint is

$$x_{4,6} + x_{5,6} = x_{6,5} + 1$$

  - This can be rewritten as

$$\sum_{j:\langle 6,j\rangle \in E} x_{6,j} - \sum_{j:\langle j,6\rangle \in E} x_{j,6} = -1$$

# Conservation of flow constraints (cont.)

- For all other nodes (neither a source or a destination), e.g. node 2, the constraint is

$$x_{1,2} + x_{4,2} = x_{2,1} + x_{2,4} + x_{2,5}$$

  - This can be rewritten as

$$\sum_{j:\langle 2,j\rangle \in E} x_{2,j} - \sum_{j:\langle j,2\rangle \in E} x_{j,2} = 0$$

- The flow conservation constraints can be written in a compact form

$$\sum_{j:\langle i,j\rangle \in E} x_{i,j} - \sum_{j:\langle j,i\rangle \in E} x_{j,i} = 0, \quad i \in N - \{s, d\}$$

# IP formulation for SPP

- SPP can be formulated as

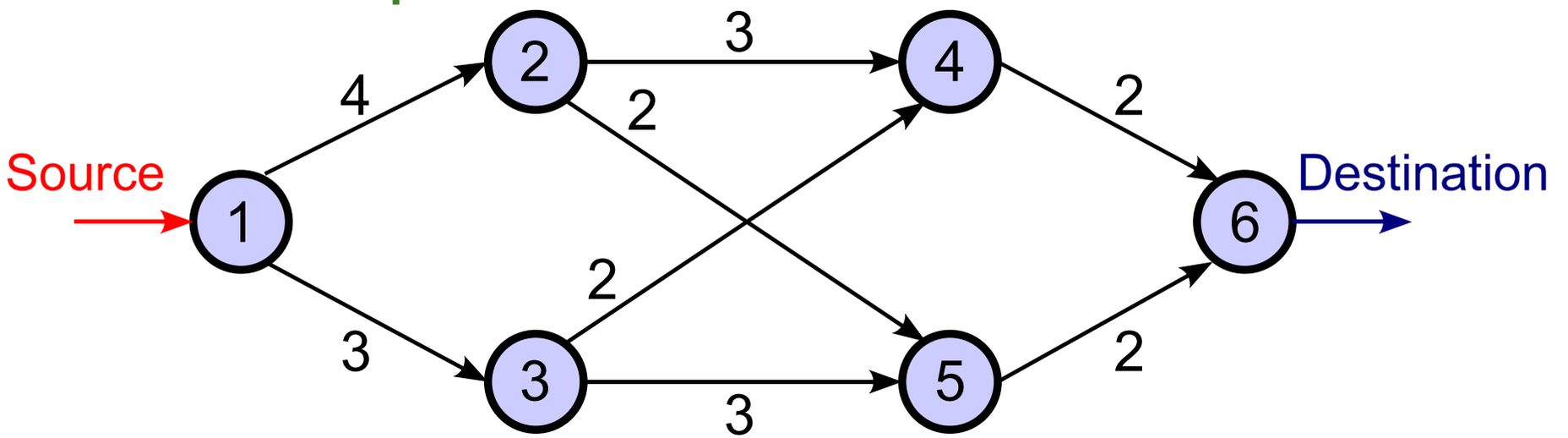
$$\min \sum_{\langle i,j \rangle \in E} c_{i,j} x_{i,j}$$

subject to

$$\sum_{j:\langle i,j \rangle \in E} x_{i,j} - \sum_{j:\langle j,i \rangle \in E} x_{j,i} = \begin{cases} 1 & \text{if } i = s \\ 0 & \text{if } i \in N - \{s, d\} \\ -1 & \text{if } i = d \end{cases}$$

$$x_{i,j} \in \{0, 1\} \text{ for all } \langle i, j \rangle \in E$$

# SPP example




- We will use AMPL/CPLEX for solving SPP in this example network
- Note:
  - It is far more efficient to use Dijkstra's algorithm for solving SPP
  - The reason of using integer programming here is for illustration only
- The files are `shortest.dat`, `shortest.mod` and `shortest_batch`

# Flow conservation constraints

- The flow conservation constraints ensure that there is only one path from the source node $s$ to destination node $d$

$$\sum_{j:\langle i,j\rangle \in E} x_{i,j} - \sum_{j:\langle j,i\rangle \in E} x_{j,i} = \begin{cases} 1 & \text{if } i = s \\ 0 & \text{if } i \in N - \{s, d\} \\ -1 & \text{if } i = d \end{cases}$$

$$x_{i,j} \in \{0, 1\} \text{ for all } \langle i, j\rangle \in E$$

# Introducing non-unit flow and link capacity (1)

(Note: A dot point preceded by ★ indicates that it is different from the setting of the shortest path problem.)

- Given
  - A directed graph $(N, E)$
  - ★ A flow of size $f$ with source node $s$ and destination node $d$
  - It costs $c_{ij}$ (per unit flow) for the flow to use directed edge $(i, j)$
  - ★ The capacity of the directed edge $(i, j)$ is $b_{ij}$
- Find which directed edges the flow should use in order that
  - The total cost is minimised
  - The entire flow must use only one path
  - ★ The flow on any directed edge does not exceed its capacity

# Introducing non-unit flow and link capacity (2)

- Decision variables are the same as before

$$x_{ij} = \begin{cases} 1 & \text{if directed edge } (i,j) \text{ is used} \\ 0 & \text{otherwise} \end{cases}$$

- The amount of flow on directed edge $(i,j)$ will be $fx_{ij}$

# Introducing non-unit flow and link capacity (3)

The problem formulation is

$$\min \sum_{(i,j)\in E} c_{ij}x_{ij}$$

subject to

$$\sum_{j:(i,j)\in E} x_{ij} - \sum_{j:(j,i)\in E} x_{ji} = \begin{cases} 1 & \text{if } i = s \\ 0 & \text{if } i \in N - \{s, d\} \\ -1 & \text{if } i = d \end{cases}$$

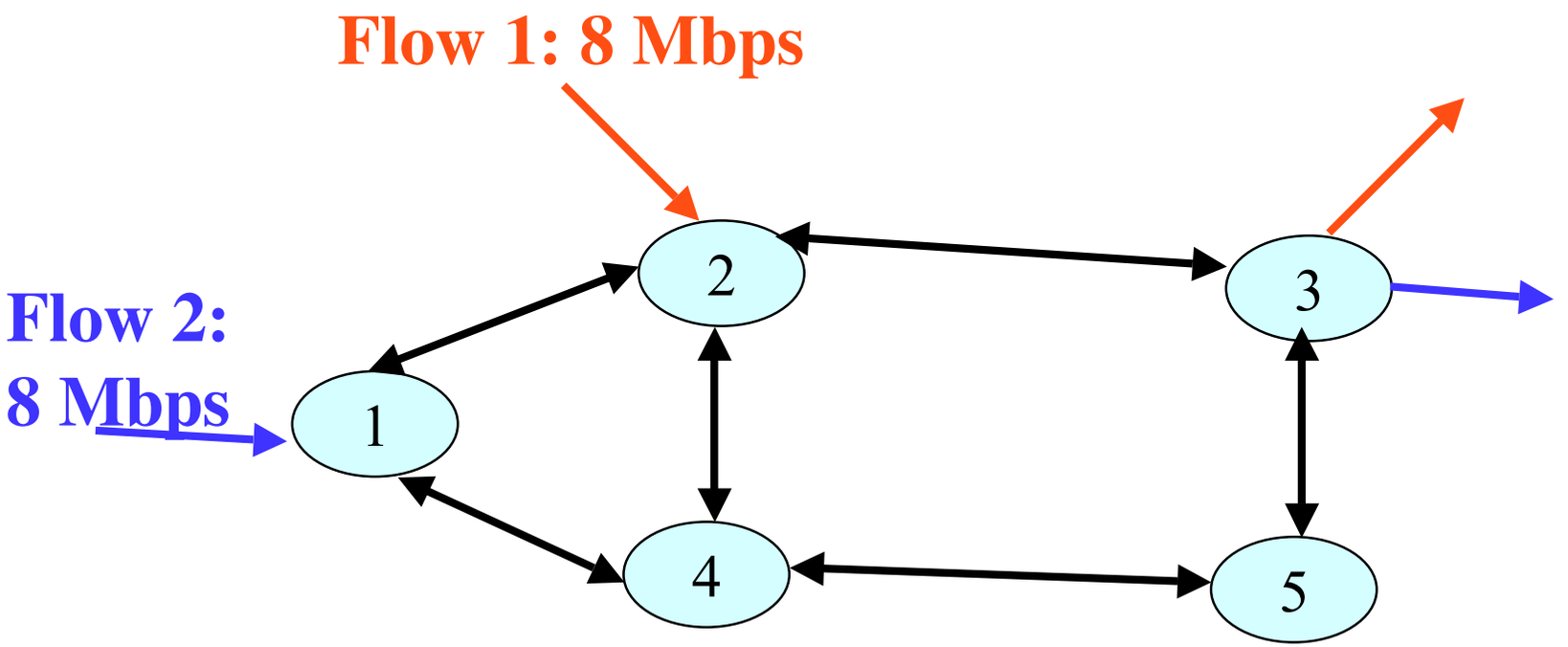
$$f x_{ij} \leq b_{ij} \text{ for all } (i,j) \in E \qquad (***)$$

$$x_{ij} \in \{0, 1\} \text{ for all } (i,j) \in E$$

Note: $(***)$ — this constraint ensures that only links with sufficient capacity may be chosen to carry the flow.
Solution: Eliminate edges with insufficient capacity, then Dijkstra.

# Multiple flows (1)

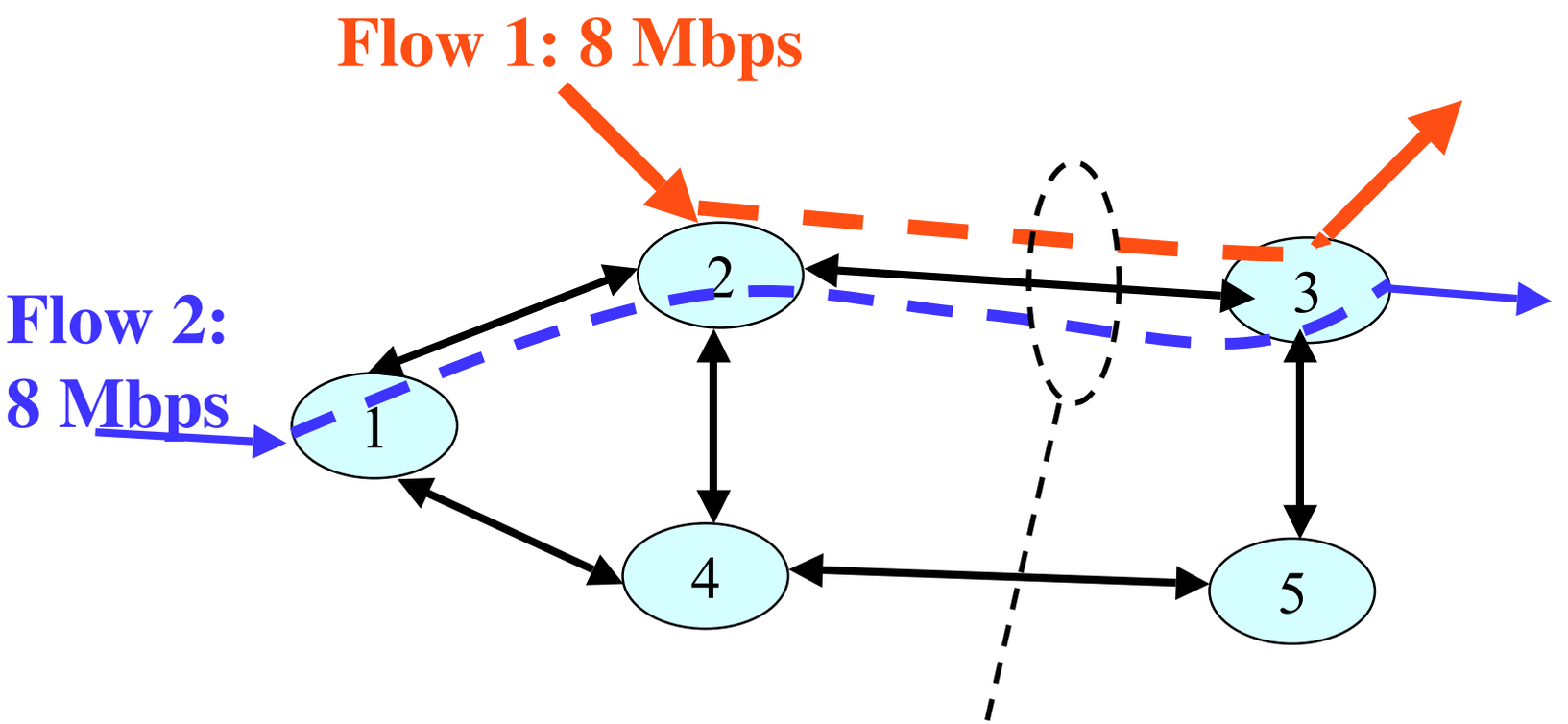
**Given: Each link has capacity of 10 Mbps**




**Assuming cost for each link is 1.**
**What if both flows use the shortest path?**

# Multiple flows (2)

**Given: Each link has capacity of 10 Mbps**




**16 Mbps of flows on 10Mbps**
**⇒ Utilisation > 1, High packet delay and loss**

# Traffic engineering problem

- Given
  - A directed graph $(N, E)$
  - ★ $m$ flows (indexed by $k = 1, 2, ...m$)
  - ★ Flow $k$ has size $f_k$, source node $s_k$, destination $d_k$
  - ★ It costs $c_{ij}$ for a unit of flow to use directed edge $(i, j)$
  - The capacity of directed edge is $b_{ij}$
- Find the directed edges that **each flow** should use in order that
  - The total cost is minimised
  - The entire flow must use only one path
  - ★ The *total flow* on a directed edge does not exceed its capacity

# Digression: Integral versus continuous traffic engineering

- There are two versions of traffic engineering problem
- The *integral* version where each flow must use only one path, i.e. all packets in a flow must use the same path
  - In order to ensure that the packets use a certain path, you can use source routing (available in IP version 6) or MPLS (multi-protocol label switching)
- The *continuous* version where each flow may use multiple paths, e.g. the one described on pages 5 – 6 of this lecture.
  - In order to split the flow, a classifier will be required at the router to send packets on different paths
- We will see how we can formulate the integral traffic engineering problem

# Traffic engineering IP formulation (1)

- Decision variables: $m$ sets of decision variables, one for *each flow*

$$x_{ijk} = \begin{cases} 1 & \text{if flow } k \text{ uses directed edge } (i,j) \\ 0 & \text{otherwise} \end{cases}$$

- The flow on directed edge $(i,j)$ will be

$$\sum_{k=1}^{m} f_k x_{ijk}$$

- Ex: $m = 3$. Flows 1 and 3 use edge (1,2) but flow 2 doesn't.

  Total flow in edge (1,2) = $f_1 + f_3$

  $\sum_{k=1}^{m} f_k x_{12k} = f_1 \times \underbrace{x_{121}}_{=1} + f_2 \times \underbrace{x_{122}}_{=0} + f_3 \times \underbrace{x_{123}}_{=1} = f_1 + f_3$

# Traffic engineering IP formulation (2)

The problem formulation is

$$\min \sum_{(i,j)\in E} \sum_{k=1}^{m} c_{ij} f_k x_{ijk}$$

subject to

$$\sum_{j:(i,j)\in E} x_{ijk} - \sum_{j:(j,i)\in E} x_{jik} = \begin{cases} 1 & \text{if } i = s_k \\ 0 & \text{if } i \in N - \{s_k, d_k\} \\ -1 & \text{if } i = d_k \end{cases} \quad k = 1, ..., m \ (*)$$

$$\sum_{k=1}^{m} f_k x_{ijk} \leq b_{ij} \quad \text{for all } (i,j) \in E \ (**)$$

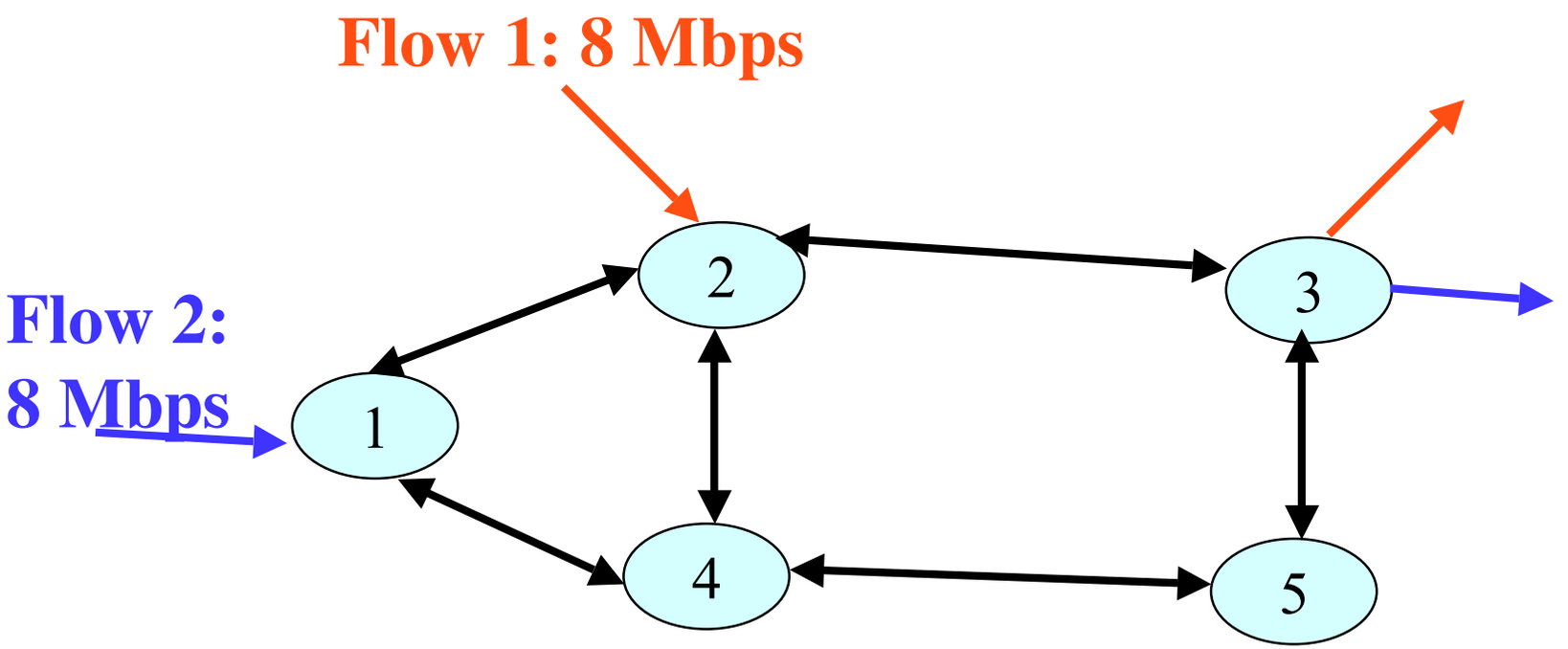
$$x_{ijk} \in \{0,1\} \text{ for all } (i,j) \in E,\ k = 1, ..., m$$

$(*)$ – One set of flow balance constraint per flow. Enforces flow $k$ is from $s_k$ to $d_k$

$(**)$ – Total flow on a link does not exceed its capacity.

# AMPL Example

**Given: Each link has capacity of 10 Mbps**




**Assuming cost for each link is 1.**
**What if both flows use the shortest path?**

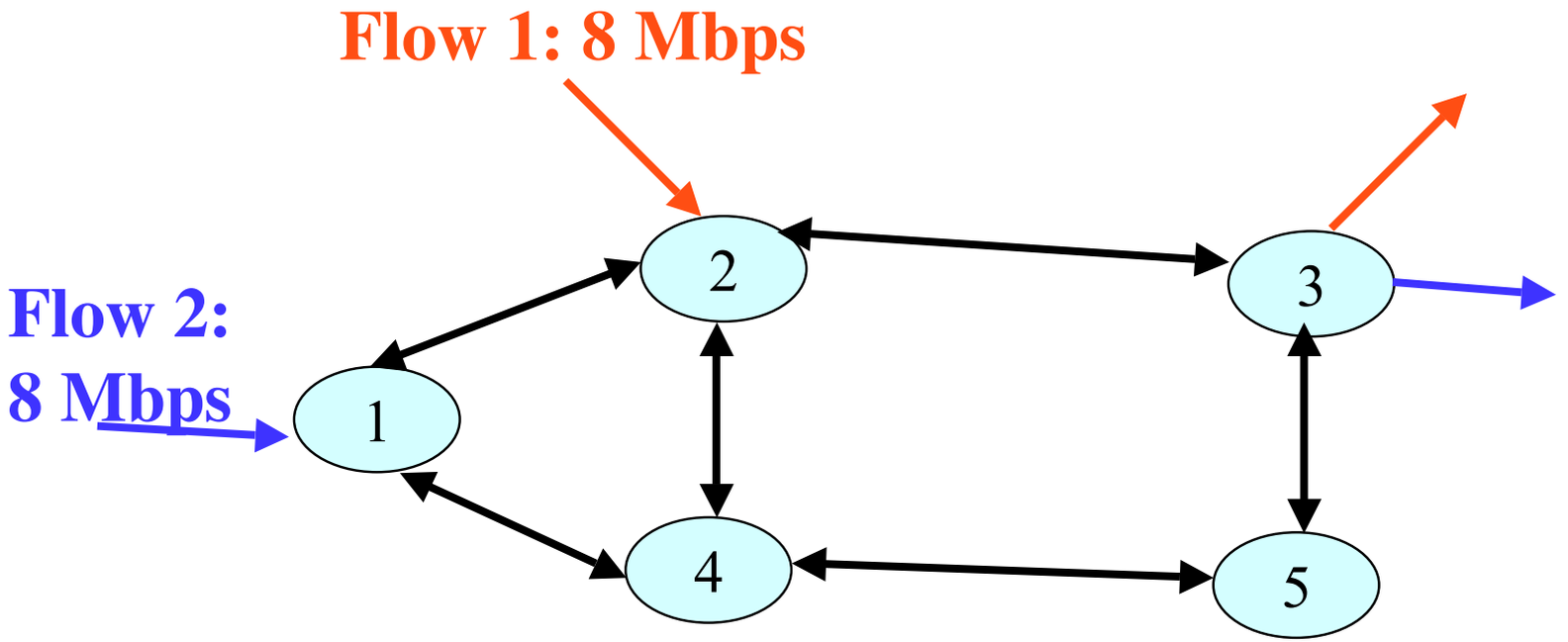
Files are `mcf1.dat`, `mcf1.mod` and `mcf1_batch`,

# Traffic engineering problem

- Also known as
  - The multi-commodity flow problem in operations research
  - Flow assignment problem
- Essence: assign a flow to a path so that performance is met
  - i.e. routing problem
- Many variations possible
  - Constraint on the path delay / number of hops
  - Constraint on packet loss rate

# Network design problem(1)

**In flow assignment, we assume the network topology and link capacities are given.**




**Why should we choose capacity 10Mbps? Why not 100Mbps? Why should we choose to have a link between (2,3) but not (2,5)?**

# Network design problem(2)

- Given
  - A set of nodes $N$
  - $m$ flows of size $f_k$, source $s_k$, destination $d_k$
  - Minimum network building cost
- Design options in network design problems
  - Topology: Which directed links to include
  - Capacity of the link
  - How the flows are routed?
- There are a few different network design problems

# Different network design problems

- Flow Assignment Problem
  - Given: flows, topology, capacity
  - Find: paths for the flows
- Capacity and Flow Assignment Problem
  - Given: flows, topology, network cost
  - Find: paths for the flows, capacity
- Topology, Capacity and Flow Assignment Problem
  - Given: flows, network cost
  - Find: paths for the flows, capacity, topology

# Summary

- Network flow problem
  - How to route traffic from a source node to destination node?
  - Many variations: From flow assignment only to topology, capacity and flow assignment
- Key new idea: Flow conservation constraint to enforce a path from a source node to a destination node

# References

- Advanced formulation of integer programming problems
  - Winston, “Operations Research”, Section 9.2
- Network flow problems
  - Ahuja et al, “Network Flows”, Sections 1.2